

Week 16 Homework: From a Linear Boundary to a Random Forest

Goal: implement and compare classic classification models, evaluate decisions clearly, and extend the workflow to linear regression.

Deliverables. Submit (1) written calculations, (2) one Python file named `week16_classic_ml.py`, (3) one classification results table, (4) one regression results table plus residual plot, (5) a short model card, and (6) the Feynman Technique post.

AI policy. You may use AI or web search to understand official NumPy, pandas, Matplotlib, or scikit-learn documentation and to interpret an error after first attempting to debug it. You may **not** ask AI to fill the required `TODO` functions, perform the hand calculations, select the winning model, write the model card, or produce the Feynman post. If you use AI, add one sentence stating exactly what it helped you understand.

Core implementation rules. Use NumPy for the manual functions. You may use scikit-learn to fit the three comparison classifiers and the linear-regression model. For the required manual metrics, do not call `sklearn.metrics` until after your own results have been printed; then use it only to check your answers. Fix all random seeds at 16.

Monday: Linear Scores and Perceptron Updates by Hand

Use labels $y \in \{-1, +1\}$ and the initial parameters

$$w = \begin{bmatrix} 0.4 \\ -0.3 \end{bmatrix}, \quad b = -0.2, \quad \eta = 0.1.$$

Process these examples in the listed order:

example	x_1	x_2	y
A	2	1	+1
B	1	3	-1
C	2	2	+1
D	-1	1	-1

For each example, calculate $z = w^T x + b$ using the *current* parameters. If $yz \leq 0$, update

$$w \leftarrow w + \eta y x, \quad b \leftarrow b + \eta y.$$

1. Make a table containing the example, old w , old b , z , yz , whether an update occurs, new w , and new b .
2. State how many mistakes or boundary cases occurred in the pass.
3. Using the final parameters, classify all four examples again.
4. Draw the initial and final boundaries on the same x_1 - x_2 axes. Label the positive and negative sides.
5. Explain why a lower number of training mistakes after one pass does not prove better test performance.

Complete these functions without scikit-learn.

```
import numpy as np

def linear_scores(X, w, b):
    # TODO: return one score per row
    pass

def signed_predictions(scores):
    # TODO: return +1 when score >= 0, otherwise -1
    pass

def perceptron_epoch(X, y, w, b, learning_rate):
    mistakes = 0
    for x_i, y_i in zip(X, y):
        # TODO: calculate score
        # TODO: update when y_i * score <= 0
        pass
    return w, b, mistakes
```

Confirm that one code epoch exactly matches your hand calculation. Avoid aliasing the original weight array when saving a history of updates; use `w.copy()`.

Tuesday: Build and Diagnose a Decision Tree

Create a reproducible binary-classification dataset using either (a) a mentor-provided transformer-inspection CSV or (b) `sklearn.datasets.make_classification` with at least 600 rows, 8 features, 3 informative features, 2 redundant features, class weights `[0.85, 0.15]`, `flip_y=0.03`, and `random_state=16`.

Use one stratified split: 60% training, 20% validation, and 20% test. The final test set must remain untouched until Friday.

1. Report the number and percentage of positive examples in each split.
2. Train decision trees with `max_depth` values 1, 2, 3, 5, 8, and unrestricted. Keep `random_state=16`.
3. For each depth, record training and validation accuracy, precision, recall, and F1 at threshold 0.5.
4. Plot training and validation F1 against depth. Identify evidence of underfitting or overfitting.
5. Select one depth using validation performance and operational reasoning. Do not use the test set.
6. Draw or export the top three levels of the selected tree. For each displayed leaf, report its class counts and predicted probability.
7. For one candidate split supplied by your mentor, manually calculate the parent Gini impurity, weighted child impurity, and impurity decrease.

In four to six sentences, explain why a deep tree can obtain excellent training performance yet change substantially when a few training rows change.

Wednesday: Build a Random Forest and Explain the Randomness

Using the same training and validation sets, train random forests for this grid:

$$n_estimators \in \{20, 100, 300\}, \quad min_samples_leaf \in \{1, 3, 8\}.$$

Use `max_features="sqrt"`, `class_weight="balanced"`, `n_jobs=-1`, and `random_state=16`.

1. Record validation precision, recall, F1, fit time, prediction time, and serialized model size for every setting.
2. Select one forest using validation results. A difference smaller than 0.01 F1 must be justified using speed, size, or recall rather than claiming a meaningful accuracy victory.
3. Compare the selected forest with Tuesday's selected tree.
4. Explain bootstrap sampling in your own words. Can one training row appear more than once in a tree's bootstrap sample?
5. Explain why considering a random feature subset at each split can improve the value of averaging.
6. Compute permutation importance on the validation set. List the five most important features and explain why the ranking does not establish causation.

Required forest diagram. Draw 5 training rows entering three different bootstrap samples, show one feature subset considered by each tree, and show the three predictions being averaged. The diagram must make both sources of randomness visible.

Thursday: Implement Classification Evaluation Clearly

Complete the following functions manually.

```
def confusion_counts(y_true, y_pred):
    tp = tn = fp = fn = 0
    for truth, prediction in zip(y_true, y_pred):
        # TODO: update exactly one count
        pass
    return tp, tn, fp, fn

def classification_metrics(tp, tn, fp, fn):
    # TODO: accuracy, precision, recall, and F1
    # Return 0 when a metric's denominator is zero.
    pass

def labels_from_probability(probabilities, threshold):
    # TODO: return an integer NumPy array
    pass
```

Use your selected forest's validation probabilities.

1. Create a table for thresholds 0.10, 0.20, ..., 0.90 containing TP, TN, FP, FN, accuracy, precision, recall, and F1.
2. Confirm every row with `sklearn.metrics` only after printing the manual results.
3. Plot precision and recall against threshold.
4. Choose a threshold for each policy:
 - (a) missing a failing transformer costs 8 times as much as an unnecessary inspection;
 - (b) the inspection team can inspect at most 20% of units;
 - (c) precision and recall are considered equally important.
5. For policy (a), calculate `cost = 8 * FN + 1 * FP` for every threshold and select the smallest validation cost.
6. Explain why the selected thresholds may differ even though the trained model and probabilities are unchanged.

Short written response. Construct an example with at least 100 cases where accuracy exceeds 95% but recall for the positive class is zero. Provide the confusion matrix and explain why the model is unacceptable for inspection screening.

Friday: Linear Regression, Final Test, and Model Card

Create a reproducible regression dataset using either a mentor-provided load CSV or this scikit-learn function:

```
sklearn.datasets.make_regression.
```

Use at least 500 rows, 6 features, noise greater than zero, and `random_state=16`. Use a fixed train/test split. If the data are chronological, use the earlier observations for training and later observations for testing.

Complete these manual metric functions.

```
def regression_metrics(y_true, y_pred):
    residuals = y_true - y_pred
    # TODO: calculate MAE
    # TODO: calculate MSE and RMSE
    # TODO: calculate R-squared using the mean of y_true
    return mae, mse, rmse, r2
```

1. Fit `LinearRegression` and predict the test targets.
2. Calculate MAE, MSE, RMSE, and R^2 manually, then verify them with scikit-learn.
3. Compare against a baseline that predicts the training-target mean for every test example.
4. Make a predicted-versus-actual scatter plot with a $y = x$ reference line.
5. Make a residual plot. Identify the five largest absolute residuals and inspect their feature values.
6. Explain which metric you would prioritize if one very large peak-load miss can trigger an expensive emergency purchase.

Now unlock the untouched classification test set. Evaluate exactly these frozen systems:

- the selected tree at its selected threshold;
- the selected forest at each of the three policy thresholds chosen Thursday; and
- a majority-class baseline.

Do not change a model or threshold after seeing these test results. Present the final classification comparison table.

Required One-Page Model Card

Write a one-page model card for the classifier you would deploy. Include:

1. intended use and positive-class definition;
2. data source, split strategy, and class balance;
3. selected model and threshold;
4. final test confusion matrix, precision, recall, F1, and error cost;
5. comparison with the baseline and rejected model;
6. latency or model-size consideration;
7. two known limitations or likely bias sources; and
8. a monitoring plan naming at least two quantities to track after deployment.

Feynman Technique Post

Create one public-facing learning note titled “**Why One Tree Is Fragile but a Random Forest Is Stable.**” It must include:

- a hand-drawn or original diagram of a linear boundary, tree regions, and a forest vote;
- a plain-language explanation of bootstrap samples and random feature subsets;
- one confusion matrix with precision and recall interpreted in words;
- one comparison between classification and regression evaluation; and
- one warning about overfitting, leakage, feature importance, or class imbalance.

Submission checklist. All random seeds are fixed; the test set was used only once; manual functions match library checks; every table labels its data split and threshold; plots have labeled axes; the model card states limitations; and any AI assistance is disclosed precisely.